DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

CSA06 – DESIGN AND ANALYSIS OF ALGORITHMS

ASSIGNMENT 3 – CO3

COMPLETE ASSIGNMENT REPORT

Q1 – Decomposition and Elimination in Ordered Data

Part A – Finding Maximum and Minimum

Q1(A): Finding Maximum and Minimum

Problem statement

Given an array of n numbers, find both the smallest and largest values using Divide and Conquer.

Algorithm design

Divide the array into two halves until a subarray has one or two elements. A one-element subarray returns the same value as both minimum and maximum. A two-element subarray needs one comparison. The parent combines two results by comparing their minimums and their maximums.

Pseudocode

```
MAXMIN(A, low, high)
    if low == high:
        return (A[low], A[low])
    if high == low + 1:
        compare A[low] and A[high]
        return (smaller, larger)
    mid = (low + high) // 2
    (min1, max1) = MAXMIN(A, low, mid)
    (min2, max2) = MAXMIN(A, mid+1, high)
    return (minimum(min1, min2), maximum(max1, max2))
```

Python implementation

```
def max_min(a, low, high):
    if low == high:
        return a[low], a[low], 0
    if high == low + 1:
        if a[low] < a[high]:
            return a[low], a[high], 1
        return a[high], a[low], 1

    mid = (low + high) // 2
    mn1, mx1, c1 = max_min(a, low, mid)
    mn2, mx2, c2 = max_min(a, mid + 1, high)
    mn = mn1 if mn1 < mn2 else mn2
    mx = mx1 if mx1 > mx2 else mx2
    return mn, mx, c1 + c2 + 2

arr = [7, 2, 9, 4, 1, 8, 3, 6]
minimum, maximum, comparisons = max_min(arr, 0, len(arr) - 1)
print("Array:", arr)
```

```
print("Minimum:", minimum)
print("Maximum:", maximum)
print("Comparisons:", comparisons)
```

Sample input

```
Array: [7, 2, 9, 4, 1, 8, 3, 6]
```

Sample output

```
Array: [7, 2, 9, 4, 1, 8, 3, 6]\nMinimum: 1\nMaximum: 9\nComparisons: 10
```

Complexity

Time: O(n), because every element is processed. For n equal to a power of two, the tournament method uses about 3n/2 − 2 comparisons. Space: O(log n) recursion stack for balanced splitting.

Analysis and observations

For eight elements, the program performs 10 comparisons. Finding the minimum and maximum independently requires (n−1)+(n−1)=14 comparisons. The divide-and-conquer method saves comparisons by finding a local minimum and maximum together. The exact count depends on whether subproblems contain one or two elements.

Comparison-count derivation

For n = 2^k and the one-or-two-element base cases, the comparison count follows C(n) = 2C(n/2) + 2 for n greater than 2, with C(2)=1. Solving gives C(n)=3n/2−2. For arbitrary n, the recurrence and base cases give a count close to this bound.

Input size n	Tournament comparisons (n=power of 2)	Independent min + max
2	1	2
4	4	6
8	10	14
16	22	30

Part B – Binary Search

Q1(B): Binary Search

Problem statement

Given a sorted array and a target value, determine whether the target exists and return its index if found.

Algorithm design

Compare the target with the middle element. If equal, stop. If the target is smaller, discard the right half; if larger, discard the left half. The discarded half cannot contain the target because the array is sorted.

Pseudocode

```
BINARY-SEARCH(A, target)
    low = 0; high = length(A)-1
    while low <= high:
        mid = (low+high)//2
        if A[mid] == target: return mid
        else if target < A[mid]: high = mid-1
        else: low = mid+1
    return NOT_FOUND
```

Python implementation

```
def binary_search(a, target):
    low, high = 0, len(a) - 1
    comparisons = 0
    while low <= high:
        mid = (low + high) // 2
        comparisons += 1
        if a[mid] == target:
            return mid, comparisons
        comparisons += 1
        if target < a[mid]:
            high = mid - 1
        else:
            low = mid + 1
    return -1, comparisons

data = [3, 8, 12, 17, 21, 26, 31, 40, 45]
for key in [26, 19]:
    index, count = binary_search(data, key)
    print("Target:", key, "Index:", index, "Comparisons:", count)
```

Sample input

```
Sorted array: [3, 8, 12, 17, 21, 26, 31, 40, 45]\nTargets: 26 and 19
```

Sample output

```
Target: 26 Index: 5 Comparisons: 4\nTarget: 19 Index: -1 Comparisons: 6
```

Complexity

Best case: O(1), when the middle element is the target. Average and worst case: O(log n), because the search interval is halved each time. Iterative space: O(1).

Analysis and observations

The sorted-order property justifies elimination. When A[mid] is greater than the target, every element to the right is also greater; when A[mid] is smaller, every element to the left is also smaller. The loop terminates when the target is found or low becomes greater than high. Binary Search must not be applied directly to an unsorted array.

Binary Search experiment

Array size	Maximum halving levels (approx.)	Worst-case order
8	3	$\log_2 8$
16	4	$\log_2 16$
32	5	$\log_2 32$
1024	10	$\log_2 1024$

Q2 – The Quality of Decomposition in Sorting (7 Marks)

Q2(A): Merge Sort

Problem statement

Sort an array in ascending order using recursive division and merging.

Algorithm design

Split the array into two nearly equal halves until each part has one element. Merge two sorted halves by repeatedly selecting the smaller front element. Merge Sort always creates balanced halves, independent of the original input order.

Pseudocode

```
MERGE-SORT(A)
    if length(A) <= 1: return A
    split A into left and right halves
    left = MERGE-SORT(left)
    right = MERGE-SORT(right)
    return MERGE(left, right)

MERGE(L, R)
    repeatedly append the smaller first remaining item
    append any remaining items
    return merged list
```

Python implementation

```
def merge_sort(a):
    if len(a) <= 1:
        return a[:]
    mid = len(a) // 2
    left = merge_sort(a[:mid])
    right = merge_sort(a[mid:])
    result = []
    i = j = 0
    while i < len(left) and j < len(right):
        if left[i] <= right[j]:
            result.append(left[i]); i += 1
        else:
            result.append(right[j]); j += 1
    result.extend(left[i:])
    result.extend(right[j:])
    return result

values = [38, 27, 43, 3, 9, 82, 10]
print("Original:", values)
print("Sorted:", merge_sort(values))
```

Sample input

```
Original: [38, 27, 43, 3, 9, 82, 10]
```

Sample output

```
Original: [38, 27, 43, 3, 9, 82, 10]\nSorted: [3, 9, 10, 27, 38, 43, 82]
```

Complexity

Time: O(n log n) in best, average and worst cases. Auxiliary space: O(n) for merging, plus recursion stack O(log n).

Analysis and observations

Merge Sort's recurrence is $T(n)=2T(n/2)+O(n)$. The two recursive calls process equal-sized halves, and merging takes linear time. It is stable when equal values from the left half are selected first. Its predictable time makes it useful when worst-case performance matters.

Q2(B): Quick Sort using last element as pivot (Lomuto partition)

Problem statement

Sort an array using partitioning around a pivot.

Algorithm design

Choose the last element as pivot. Move values less than or equal to the pivot to its left, then place the pivot in its final position. Recursively sort the left and right partitions. Partition quality depends on the pivot and input order.

Pseudocode

```
QUICK-SORT(A, low, high)
    if low < high:
        p = PARTITION(A, low, high)
        QUICK-SORT(A, low, p-1)
        QUICK-SORT(A, p+1, high)

PARTITION(A, low, high)
    pivot = A[high]
    boundary = low-1
    for j from low to high-1:
        if A[j] <= pivot:
            boundary = boundary+1
            swap A[boundary], A[j]
    swap A[boundary+1], A[high]
    return boundary+1
```

Python implementation

```
def quick_sort(a):
    def partition(low, high):
        pivot = a[high]
        i = low - 1
        for j in range(low, high):
            if a[j] <= pivot:
                i += 1
                a[i], a[j] = a[j], a[i]
        a[i + 1], a[high] = a[high], a[i + 1]
        return i + 1

    def sort(low, high):
        if low < high:
            p = partition(low, high)
            sort(low, p - 1)
            sort(p + 1, high)

    sort(0, len(a) - 1)
    return a

data = [29, 10, 14, 37, 13, 5]
print("Sorted:", quick_sort(data))
```

Sample input

```
Input: [29, 10, 14, 37, 13, 5]
```

Sample output

```
Sorted: [5, 10, 13, 14, 29, 37]
```

Complexity

Best and average time: O(n log n). Worst time: $O(n^2)$, when partitions are repeatedly very unbalanced. Auxiliary recursion space: O(log n) for balanced partitions and O(n) in the worst case. The shown implementation sorts in place.

Analysis and observations

With a last-element pivot, already sorted or reverse-sorted input can repeatedly produce partitions of sizes 0 and n−1. The recursion depth becomes n and the work becomes quadratic. Balanced partitions of approximately n/2 and n/2 produce a recursion-tree height of O(log n). Random or median-based pivot selection can reduce the chance of consistently poor splits but does not change the theoretical worst-case bound of this implementation.

Partition-quality experiment

Input pattern	Typical partition pattern with last pivot	Recursion depth trend	Time trend
Random / favourable	Often more balanced	Near log n	Often O(n log n)
Already increasing	0 and n−1 for this pivot rule	Near n	$O(n^2)$
Decreasing	Often highly unbalanced	Can approach n	Can approach $O(n^2)$

Joint design question

Before choosing a sorting method for unknown input, we would want to know the expected input size, whether data may already be sorted or contain many duplicates, whether stable ordering is required, available memory, and whether predictable worst-case time is important. Merge Sort is a safer choice for predictable O(n log n) time and stability, but it needs extra memory. Quick Sort is often fast in practice and can sort in place, but pivot selection and input characteristics affect its performance. A randomized or three-way partitioning Quick Sort can improve practical robustness.

Q3 – Breaking the Quadratic Barrier: Karatsuba Multiplication (7 Marks)

Problem statement

Multiply two large non-negative integers. Compare the ordinary digit-based method with Karatsuba's recursive method.

Algorithm design

Write x=a·10^m+b and y=c·10^m+d. Ordinary divide-and-conquer needs four half-size multiplications: ac, ad, bc and bd. Karatsuba computes z2=ac, z0=bd, and z1=(a+b)(c+d)−z2−z0. The answer is z2·10^(2m)+z1·10^m+z0, using only three recursive multiplications.

Pseudocode

```
KARATSUBA(x, y)
    if x or y is small: return x*y
    split x into a,b and y into c,d using base 10^m
    z2 = KARATSUBA(a,c)
    z0 = KARATSUBA(b,d)
    z1 = KARATSUBA(a+b,c+d) - z2 - z0
    return z2*10^(2m) + z1*10^m + z0
```

Python implementation

```
def karatsuba(x, y):
    if x < 10 or y < 10:
        return x * y
    n = max(len(str(x)), len(str(y)))
    m = n // 2
    base = 10 ** m
    a, b = divmod(x, base)
    c, d = divmod(y, base)
    z2 = karatsuba(a, c)
    z0 = karatsuba(b, d)
    z1 = karatsuba(a + b, c + d) - z2 - z0
    return z2 * (10 ** (2 * m)) + z1 * base + z0

x, y = 1234, 5678
print("Standard result:", x * y)
print("Karatsuba result:", karatsuba(x, y))
```

Sample input

```
x = 1234\ny = 5678
```

Sample output

```
Standard result: 7006652\nKaratsuba result: 7006652
```

Complexity

Standard multiplication of two n-digit numbers takes $O(n^2)$ digit operations. Karatsuba follows $T(n)=3T(n/2)+O(n)$, giving $O(n^{\log_2 3}) \approx O(n^{1.585})$. Recursion space is O(log n), excluding integer representation overhead.

Analysis and observations

The reduction from four recursive multiplications to three improves asymptotic complexity. However, Karatsuba performs extra additions, subtractions, splitting and recursive-call overhead. For small integers, ordinary multiplication can be faster. The crossover depends on the implementation, hardware and integer size; it should be measured rather than assumed.

Recurrence derivation

Karatsuba makes three subproblems of approximately n/2 digits and performs O(n) additional work to add, subtract and combine the partial results. Therefore T(n)=3T(n/2)+cn. By the Master Theorem, n^($\log_2 3$) dominates n, so T(n)=Θ(n^1.585).

Crossover experiment design

Test increasing digit lengths, for example 8, 16, 32, 64, 128, 256 and 512 digits. For each size, generate operands of the same digit length, run each method many times, and compare the median elapsed time. Record the Python version and computer details. Python's built-in integer multiplication is highly optimized, so it is not a fair direct implementation of schoolbook multiplication for benchmarking; the sample above verifies the result, while a controlled benchmark should implement the schoolbook algorithm explicitly.

Illustrative operation comparison (not measured timings)

Digit length n	Schoolbook growth n²	Karatsuba growth n^1.585 (approx.)
16	256	~84
32	1,024	~252
64	4,096	~760
128	16,384	~2,290

The values are growth indicators, not measured CPU operation counts or timing results. The practical crossover must be established experimentally.

Q4 – The Reality of Strassen's Matrix Multiplication (7 Marks)

Problem statement

Multiply two square matrices and compare conventional multiplication with Strassen's divide-and-conquer method.

Algorithm design

Conventional block multiplication computes eight products of half-size blocks. Strassen uses seven products and additional matrix additions/subtractions. For 2×2 block matrices A=[[a,b],[c,d]] and B=[[e,f],[g,h]], define seven products and combine them to obtain the four output blocks. If n is not a power of two, pad matrices with zero rows and columns to the next power of two, then crop the result.

Pseudocode

```
STRASSEN(A, B)
    if matrix size is small:
        return conventional multiplication
    split A into a,b,c,d and B into e,f,g,h
    M1=(a+d)(e+h)
    M2=(c+d)e
    M3=a(f-h)
    M4=d(g-e)
    M5=(a+b)h
    M6=(c-a)(e+f)
    M7=(b-d)(g+h)
    C11=M1+M4-M5+M7
    C12=M3+M5
    C21=M2+M4
    C22=M1-M2+M3+M6
    combine C blocks and return
```

Python implementation

```
def add(A, B):
    n = len(A)
    return [[A[i][j] + B[i][j] for j in range(n)] for i in range(n)]

def sub(A, B):
    n = len(A)
    return [[A[i][j] - B[i][j] for j in range(n)] for i in range(n)]

def conventional(A, B):
    n = len(A)
    C = [[0] * n for _ in range(n)]
    for i in range(n):
        for k in range(n):
            for j in range(n):
                C[i][j] += A[i][k] * B[k][j]
    return C

def strassen(A, B):
    n = len(A)
    if n <= 2:
        return conventional(A, B)
    m = n // 2
    a = [row[:m] for row in A[:m]]
    b = [row[m:] for row in A[:m]]
    c = [row[:m] for row in A[m:]]
    d = [row[m:] for row in A[m:]]
    e = [row[:m] for row in B[:m]]
    f = [row[m:] for row in B[:m]]
    g = [row[:m] for row in B[m:]]
```

```
    h = [row[m:] for row in B[m:]]

    M1 = strassen(add(a, d), add(e, h))
    M2 = strassen(add(c, d), e)
    M3 = strassen(a, sub(f, h))
    M4 = strassen(d, sub(g, e))
    M5 = strassen(add(a, b), h)
    M6 = strassen(sub(c, a), add(e, f))
    M7 = strassen(sub(b, d), add(g, h))

    C11 = add(sub(add(M1, M4), M5), M7)
    C12 = add(M3, M5)
    C21 = add(M2, M4)
    C22 = add(add(sub(M1, M2), M3), M6)

    top = [C11[i] + C12[i] for i in range(m)]
    bottom = [C21[i] + C22[i] for i in range(m)]
    return top + bottom

def pad_square(M, size):
    old = len(M)
    return [row + [0] * (size-old) for row in M] + [[0]*size for _ in range(size-old)]

def strassen_any_size(A, B):
    n = len(A)
    size = 1
    while size < n:
        size *= 2
    Ap, Bp = pad_square(A, size), pad_square(B, size)
    Cp = strassen(Ap, Bp)
    return [row[:n] for row in Cp[:n]]

A = [[1, 2], [3, 4]]
B = [[5, 6], [7, 8]]
print("Conventional:", conventional(A, B))
print("Strassen:", strassen_any_size(A, B))
```

Sample input

```
A = [[1, 2], [3, 4]]\nB = [[5, 6], [7, 8]]
```

Sample output

```
Conventional: [[19, 22], [43, 50]]\nStrassen: [[19, 22], [43, 50]]
```

Complexity

Conventional multiplication: $O(n^3)$ time. Strassen: $T(n)=7T(n/2)+O(n^2)$, so $O(n^{\log_2 7}) \approx O(n^{2.807})$. Matrix storage and temporary matrices require additional memory; the exact peak memory depends on implementation.

Analysis and observations

Strassen reduces recursive block multiplications from eight to seven, but increases additions, subtractions, temporary allocations and data movement. For small matrices, conventional multiplication is usually simpler and can be faster. Padding a non-power-of-two matrix can increase the processed matrix size and waste work. The supplied implementation uses a small conventional base case and pads square matrices to a power of two.

Correctness and engineering notes

The seven products are algebraically combined to reproduce the four conventional output blocks. The implementation should be tested against conventional multiplication for several matrix sizes, including odd sizes, negative entries and

zero matrices. For rectangular matrices, additional padding to a common square size or a separate rectangular strategy is needed; the supplied wrapper assumes square matrices.

Sample verification cases

Matrix size	Test	Expected verification
1×1	[[3]] × [[4]]	[[12]]
2×2	Matrices shown above	[[19,22],[43,50]]
3×3	Any integer matrices	Strassen output equals conventional output after padding

Joint engineering decision

Use conventional multiplication for small matrices, when memory is limited, or when implementation simplicity and low overhead matter. Consider Strassen for sufficiently large dense matrices when the implementation has been tested and measured on the target hardware. The asymptotically faster method is not automatically faster at every size.

Q5 – Upgrading Closest Pair Using Divide and Conquer (6 Marks)

Problem statement

Given points in a two-dimensional plane, find the pair with the smallest Euclidean distance.

Algorithm design

Brute force checks every pair and takes $O(n^2)$. Divide and Conquer sorts points by x-coordinate, splits them into left and right halves, recursively finds the best distance in each half, and then checks points in a vertical strip around the dividing line. The strip is sorted by y-coordinate; geometric packing limits how many following points need to be checked for each point.

Pseudocode

```
CLOSEST-PAIR(P)
    sort points by x-coordinate
    if number of points <= 3:
        return brute-force closest pair
    divide P into left and right halves
    best_left = CLOSEST-PAIR(left)
    best_right = CLOSEST-PAIR(right)
    delta = smaller distance from the two halves
    strip = points whose x-distance from dividing line < delta
    sort strip by y-coordinate
    compare each strip point with only a bounded number
        of following points in y-order
    return the closest pair found
```

Python implementation

```
import math

def distance(p, q):
    return math.hypot(p[0] - q[0], p[1] - q[1])

def brute_closest(points):
    best = (None, None, float("inf"))
    for i in range(len(points)):
        for j in range(i + 1, len(points)):
            d = distance(points[i], points[j])
            if d < best[2]:
                best = (points[i], points[j], d)
    return best

def closest_pair(points):
    px = sorted(points, key=lambda p: (p[0], p[1]))
    py = sorted(points, key=lambda p: (p[1], p[0]))

    def solve(xs, ys):
        n = len(xs)
        if n <= 3:
            return brute_closest(xs)
        mid = n // 2
        left_x, right_x = xs[:mid], xs[mid:]
        left_set = set(left_x)
        left_y = [p for p in ys if p in left_set]
        right_y = [p for p in ys if p not in left_set]
        dl = solve(left_x, left_y)
        dr = solve(right_x, right_y)
        best = dl if dl[2] <= dr[2] else dr
        mid_x = xs[mid][0]
        strip = [p for p in ys if abs(p[0] - mid_x) < best[2]]
```

```
            for i in range(len(strip)):
                j = i + 1
                while j < len(strip) and strip[j][1] - strip[i][1] < best[2]:
                    d = distance(strip[i], strip[j])
                    if d < best[2]:
                        best = (strip[i], strip[j], d)
                    j += 1
            return best

        if len(points) < 2:
            raise ValueError("At least two points are required")
        return solve(px, py)

pts = [(2, 3), (12, 30), (40, 50), (5, 1), (12, 10), (3, 4)]
p1, p2, d = closest_pair(pts)
print("Closest points:", p1, p2)
print("Distance:", round(d, 4))
print("Brute-force distance:", round(brute_closest(pts)[2], 4))
```

Sample input

```
Points: [(2, 3), (12, 30), (40, 50), (5, 1), (12, 10), (3, 4)]
```

Sample output

```
Closest points: (2, 3) (3, 4)\nDistance: 1.4142\nBrute-force distance: 1.4142
```

Complexity

Brute force: $O(n^2)$ time and O(1) extra space beyond input. Standard Divide-and-Conquer closest pair: O(n log n) time when sorting and strip processing are implemented efficiently; recursion uses O(log n) stack. This educational implementation sorts initially and partitions y-lists at each recursive level; its asymptotic target is O(n log n), with extra list/set storage.

Analysis and observations

The strip is not compared against every other point. Points are ordered by y-coordinate, and geometric separation guarantees that only a constant number of following points can be relevant before the y-distance is at least the current best distance. The algorithm handles the cross-boundary pair that could be closer than either half's answer. The returned distance is checked against brute force in the sample.

Comparison plan

Use the same randomly generated point sets for both algorithms. For each n (for example 10, 100, 500, 1,000 and 5,000), record distance calculations and median runtime over repeated runs. The brute-force count is exactly $n(n-1)/2$. The divide-and-conquer count should grow much more slowly for large n. Do not report machine timings unless they are measured on the actual system.

n points	Brute-force pair checks $n(n-1)/2$	Expected growth comparison
10	45	Both are small
100	4,950	D&C begins reducing pair work
500	124,750	Quadratic work grows rapidly
1,000	499,500	D&C expected to scale better

Q6 – Advanced Divide-and-Conquer Mini-Investigations (7 Marks)

Q6(A): Median of Medians – k-th Smallest Element

Problem statement

Find the k-th smallest element in an unsorted list with a deterministic worst-case linear-time selection algorithm.

Algorithm design

Divide the values into groups of at most five and find each group's median. Recursively find the median of these medians and use it as a pivot. Partition the input around the pivot and recurse only into the side containing the required rank. Groups of five guarantee a sufficiently balanced pivot in the worst case.

Pseudocode

```
SELECT(A, k)
    if A has at most 5 elements:
        sort A and return element at index k
    divide A into groups of five
    find each group median
    pivot = SELECT(medians, median index)
    divide A into values less than, equal to, and greater than pivot
    if k is in lower group: recurse there
    else if k is in equal group: return pivot
    else recurse in upper group with adjusted k
```

Python implementation

```
def median_of_medians_select(a, k):
    # k is a 1-based rank
    if not 1 <= k <= len(a):
        raise ValueError("k is out of range")
    if len(a) <= 5:
        return sorted(a)[k - 1]

    groups = [a[i:i+5] for i in range(0, len(a), 5)]
    medians = [sorted(group)[len(group)//2] for group in groups]
    pivot = median_of_medians_select(medians, (len(medians)+1)//2)

    lower = [x for x in a if x < pivot]
    equal = [x for x in a if x == pivot]
    upper = [x for x in a if x > pivot]

    if k <= len(lower):
        return median_of_medians_select(lower, k)
    if k <= len(lower) + len(equal):
        return pivot
    return median_of_medians_select(upper, k-len(lower)-len(equal))

values = [12, 3, 5, 7, 4, 19, 26, 1, 8, 15, 2]
k = 5
print("Values:", values)
print(k, "th smallest:", median_of_medians_select(values, k))
```

Sample input

```
Values: [12, 3, 5, 7, 4, 19, 26, 1, 8, 15, 2]\nk = 5
```

Sample output

```
Values: [12, 3, 5, 7, 4, 19, 26, 1, 8, 15, 2]\n5 th smallest: 5
```

Complexity

Deterministic worst-case time: O(n). The recurrence is bounded by $T(n) \leq T(n/5) + T(\text{approximately } 7n/10) + O(n)$, which resolves to O(n). Auxiliary space in this straightforward Python list implementation is O(n) due to created groups and partitions; recursion depth is logarithmic in the shrinking subproblem sizes.

Analysis and observations

Pivot quality is central to selection. A very poor pivot leaves almost the entire array for the next call and can cause $O(n^2)$ QuickSelect behaviour. Median of Medians guarantees a pivot that discards a constant fraction of the elements, ensuring worst-case linear time. Randomized QuickSelect has expected O(n) time but still has $O(n^2)$ worst-case time.

QuickSelect comparison

```
def quickselect_randomized(a, k):
    # k is a 1-based rank; expected O(n), worst O(n^2)
    import random
    if not 1 <= k <= len(a):
        raise ValueError("k is out of range")
    arr = a[:]
    while True:
        if len(arr) == 1:
            return arr[0]
        pivot = random.choice(arr)
        low = [x for x in arr if x < pivot]
        equal = [x for x in arr if x == pivot]
        high = [x for x in arr if x > pivot]
        if k <= len(low):
            arr = low
        elif k <= len(low) + len(equal):
            return pivot
        else:
            k -= len(low) + len(equal)
            arr = high
```

A deterministic poor-pivot strategy such as always selecting the first element can be particularly bad on already sorted input. Randomization reduces the likelihood of repeatedly choosing an extreme pivot, but it does not remove the theoretical worst case.

Q6(B): Meet-in-the-Middle – Subset Sum

Problem statement

Determine whether a subset of the given numbers adds exactly to a target value.

Algorithm design

Split the n values into two halves. Generate all subset sums for each half. For each sum from the first half, search for target−sum in the second-half sums. This reduces the time from O(2^n) enumeration to approximately O(2^(n/2)) subset generation and lookup, at the cost of O(2^(n/2)) memory.

Pseudocode

```
MEET-IN-THE-MIDDLE(A, target)
    split A into left and right halves
    L = all subset sums of left
    R = all subset sums of right
    store R in a hash set
    for each x in L:
        if target-x exists in R: return TRUE
    return FALSE
```

Python implementation

```
def subset_sums(values):
    sums = [0]
    for value in values:
        sums += [s + value for s in sums]
    return sums


def meet_in_middle_subset_sum(values, target):
    mid = len(values) // 2
    left_sums = subset_sums(values[:mid])
    right_sums = set(subset_sums(values[mid:]))
    for s in left_sums:
        if target - s in right_sums:
            return True
    return False


numbers = [3, 34, 4, 12, 5, 2]
target = 9
print("Numbers:", numbers)
print("Target:", target)
print("Subset exists:", meet_in_middle_subset_sum(numbers, target))
```

Sample input

```
Numbers: [3, 34, 4, 12, 5, 2]\nTarget: 9
```

Sample output

```
Numbers: [3, 34, 4, 12, 5, 2]\nTarget: 9\nSubset exists: True
```

Complexity

Time: O(2^(n/2)) expected with hash-set lookup, ignoring hash collisions and output construction. Space: O(2^(n/2)). Straight enumeration takes O(2^n) time and usually much less extra memory if subsets are processed one at a time.

Analysis and observations

The technique trades memory for reduced search time. It is useful for moderate n when 2^n is too large but 2^(n/2) storage is manageable. For example, at n=40, full enumeration has about 1,099,511,627,776 subsets, while each half has 1,048,576 subsets. Negative and zero values are supported by this implementation.

Time-memory comparison

n	Full enumeration 2^n	Half enumeration 2^(n/2) per half
20	1,048,576	1,024
30	1,073,741,824	32,768
40	1,099,511,627,776	1,048,576
50	1,125,899,906,842,624	33,554,432

Q7 – Design Divide and Conquer from First Principles (10 Marks)

Problem statement

Designed problem: Given an array of integers, find the maximum sum of any contiguous, non-empty subarray. This is a suitable first-principles Divide-and-Conquer challenge because the best subarray is either entirely in the left half, entirely in the right half, or crosses the midpoint.

Algorithm design

Each recursive call returns the best subarray sum within its interval. The base case is a single element. The combine step computes the best suffix sum of the left half and the best prefix sum of the right half; their sum is the best crossing subarray. The final answer is the maximum of left, right and crossing sums.

Pseudocode

```
MAX-SUBARRAY(A, low, high)
    if low == high: return A[low]
    mid = (low+high)//2
    left_best = MAX-SUBARRAY(A, low, mid)
    right_best = MAX-SUBARRAY(A, mid+1, high)
    left_suffix = maximum sum ending at mid
    right_prefix = maximum sum starting at mid+1
    cross_best = left_suffix + right_prefix
    return max(left_best, right_best, cross_best)

MAX-SUFFIX(A, low, mid)
    scan from mid down to low, keep maximum running sum
    return maximum

MAX-PREFIX(A, mid+1, high)
    scan from mid+1 up to high, keep maximum running sum
    return maximum
```

Python implementation

```
def max_subarray_divide_conquer(a):
    if not a:
        raise ValueError("Array must not be empty")

    def solve(low, high):
        if low == high:
            return a[low]

        mid = (low + high) // 2
        left_best = solve(low, mid)
        right_best = solve(mid + 1, high)

        total = 0
        left_suffix = float("-inf")
        for i in range(mid, low - 1, -1):
            total += a[i]
            left_suffix = max(left_suffix, total)

        total = 0
        right_prefix = float("-inf")
        for i in range(mid + 1, high + 1):
            total += a[i]
            right_prefix = max(right_prefix, total)

        cross_best = left_suffix + right_prefix
        return max(left_best, right_best, cross_best)
```

```
    return solve(0, len(a) - 1)

data = [-2, 1, -3, 4, -1, 2, 1, -5, 4]
print("Array:", data)
print("Maximum contiguous subarray sum:", max_subarray_divide_conquer(data))
```

Sample input

```
Array: [-2, 1, -3, 4, -1, 2, 1, -5, 4]
```

Sample output

```
Array: [-2, 1, -3, 4, -1, 2, 1, -5, 4]\nMaximum contiguous subarray sum: 6
```

Complexity

Time: $T(n)=2T(n/2)+O(n)$, so $O(n \log n)$. Recursion stack: $O(\log n)$. The algorithm handles arrays containing all negative values because the base case returns the element itself and prefix/suffix scans begin at negative infinity.

Analysis and observations

The best contiguous subarray is either wholly in one half or crosses the midpoint; these three cases cover all possibilities. The combine work is linear because the algorithm scans outward from the midpoint to obtain the best left suffix and right prefix. A straightforward baseline checks every possible subarray in $O(n^2)$ time. The divide-and-conquer method improves the asymptotic time to $O(n \log n)$.

Stage 1 – Independent design proposals (sample records)

Student A proposal: Split the array into two halves. Recursively find the best sum in each half. For a crossing answer, scan from the middle toward the left and from the middle toward the right, then add the two best partial sums.

Student B proposal: Divide the array into left, right and crossing possibilities. Use a one-element base case. Return the maximum among the left answer, right answer and crossing sum.

Stage 2 – Design comparison

Aspect	Student A proposal	Student B proposal	Final decision
Decomposition	Two equal halves	Left, right and crossing cases	Split into two halves and evaluate all three answer locations
Base case	One element	One element	Keep one-element base case
Combine	Find best suffix and prefix	Compare left/right/crossing	Use both proposals together
Correctness	Crossing scan captures boundary-spanning result	Three cases cover every possible answer	Retain the complete three-case proof

Stage 3 – Final design and correctness

- Decomposition: split the current interval at its midpoint.
- Subproblems: maximum subarray sum in the left and right intervals.
- Base case: one element; that element is the only non-empty subarray.
- Returned information: the best sum for the interval.
- Combine: calculate the maximum suffix ending at the midpoint and maximum prefix starting immediately after it; add them to form the best crossing sum.
- Correctness: every contiguous subarray lies wholly in the left half, wholly in the right half, or crosses the midpoint. The algorithm evaluates the best possibility in each category.

Stage 4 – Recurrence and complexity

There are two recursive calls on approximately n/2 elements. The crossing scan takes O(n) time. Therefore $T(n)=2T(n/2)+cn$, with $T(1)=\Theta(1)$. By the Master Theorem, $T(n)=\Theta(n \log n)$.

Stage 5 – Baseline comparison

A brute-force baseline enumerates every start and end index and calculates each subarray sum. With a running sum inside the nested loops, it performs $\Theta(n^2)$ additions. For an experimental comparison, use the same random arrays for both methods, test increasing n, verify that answers match, and record operation counts and repeated median runtimes.

n	Brute-force candidate subarrays $n(n+1)/2$	D&C growth indicator $n \log_2 n$
8	36	24
16	136	64
32	528	160
64	2,080	384
128	8,256	896

The D&C column is a growth indicator, not an exact count of Python operations. Actual timings should be measured on the machine used for submission.

Stage 6 – Joint evaluation

Divide and Conquer is appropriate when a problem can be split into smaller instances, the subproblem answers are useful for constructing the original answer, and the recursive work is less than solving the whole problem directly. It reduces the maximum-subarray search from quadratic to n log n time. Its costs include recursion, repeated scanning and function-call overhead. For small arrays, a simple iterative algorithm may be faster. Designers should inspect whether subproblems are independent, whether the combine step is efficient, how balanced the split is, and whether recursion introduces excessive overhead.

Overall Conclusion

This assignment demonstrates how Divide and Conquer solves problems by dividing an input into smaller subproblems, solving them recursively and combining their results. The experiments and analyses show that decomposition quality strongly affects performance. Merge Sort has predictable O(n log n) time, while Quick Sort can reach $O(n^2)$ with repeatedly unbalanced partitions. Karatsuba and Strassen reduce the number of recursive multiplications and improve asymptotic complexity, but their overhead can make them unsuitable for small inputs. The closest-pair, Median of Medians and Meet-in-the-Middle methods demonstrate how careful combining, pivot selection and time-memory trade-offs improve performance. Divide and Conquer should be selected after considering input size, balance, memory, combine cost and practical overhead.

References

- Course material: CSA06 – Design and Analysis of Algorithms, Assignment 3 – CO3 (provided assignment brief).
- Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest and Clifford Stein, Introduction to Algorithms, MIT Press.
- Jon Kleinberg and Éva Tardos, Algorithm Design, Pearson.
- Python 3 documentation, https://docs.python.org/3/ (for Python language syntax and standard library reference).

Submission checklist

- Fill in both students' names, register numbers, class and submission date.
- Run each program in Python 3 and verify the displayed sample output.
- Replace illustrative growth tables with measured experimental data if your faculty requires actual timing evidence.
- Attach screenshots of program execution and any additional datasets or graphs.
- Both students should prepare to explain all seven questions, including the partner's primary sections.